

Logic

Unit 1-1 Lecture Notes

MSDS 520: Math Fundamentals for Data Science
Meharry Medical College

These notes are the written companion to the Unit 1-1 slides and follow them section by section. They go further than the slides: every idea gets a fuller explanation and several worked examples, many of them drawn from clinical data, and most sections end with extra practice. Every exercise is followed by a worked solution; try each one before reading on.

Contents

1	Why Logic?	1
2	Statements	3
3	Connectives and Truth Tables	7
4	Compound Statements	21
5	De Morgan's Laws	34
6	Converse, Inverse, and Contrapositive	40
7	Review	46

1 Why Logic?

1.1 A Problem to Motivate Us

Suppose you are running a study and need to say **exactly** who is eligible to take part. The protocol reads:

“Patients who are diabetic AND (hypertensive OR obese), but NOT pregnant.”

This one sentence has two jobs. It has to be precise enough that two people reading it pick out the **same** patients, and precise enough that a **computer** can pick them out for you from a database of thousands of records. Ordinary English is not that precise: words like “or” and “but” carry shades of meaning that two readers can take differently. Logic is the language that removes that

ambiguity. By the end of these notes we will be able to write the criterion above as a formula with exactly one meaning, and to negate it correctly to find the patients who are *not* eligible.

To see that the worry is real, here are three places where a careful reader could misread even this short criterion.

- **Grouping.** Drop the parentheses and it reads “diabetic and hypertensive or obese.” Does an obese patient without diabetes qualify? One reader hears “diabetic and hypertensive, or else obese” and says yes; another hears “diabetic, and also hypertensive or obese” and says no. Section 4 shows that these really are two different cohorts.
- **The word “or.”** Is a patient who is *both* hypertensive and obese still eligible? In everyday speech “or” sometimes means “one or the other, but not both.” Section 3 settles which meaning mathematics uses.
- **The word “but.”** “But NOT pregnant” sounds like a contrast, yet logically all it does is add one more requirement. We will see that “but” is simply AND.

Each of these is a spot where two careful people, or a person and a computer, could disagree without either one noticing. Logic gives each word one fixed meaning, so that disagreement cannot happen.

1.2 Where This Shows Up in Data Science

The words of logic (AND, OR, NOT, IF-THEN) appear everywhere in data work:

- **Filtering data:** keeping the rows of a table that satisfy a condition, such as “A1c at least 6.5%.”
- **Database and EHR queries:** a query such as WHERE diabetic AND NOT pregnant is a logical statement.
- **Model decisions:** a classifier ultimately answers a true/false question about each input: does this X-ray show pneumonia, should this sepsis alert fire.
- **Proofs and definitions:** every definition in this course is a logical statement.

Worked example: a filter is one truth value per row

Suppose a study keeps the adult patients whose A1c is at least 6.5%. The filter asks two yes/no questions about each row of the table and keeps the row only when both answers are yes.

Patient	Age	A1c (%)	Age $\geq$ 18?	A1c $\geq$ 6.5?	Keep?
A	34	7.1	T	T	T
B	16	8.0	F	T	F
C	58	5.9	T	F	F
D	71	6.5	T	T	T

Patient B has a high A1c but is excluded for age, and patient C is an adult whose A1c is too low. Patient D is kept even though the A1c is exactly 6.5, because “at least” includes the boundary

value. Every row gets exactly one answer, T or F, and the filter keeps the T rows. Everything in these notes is, in the end, about computing that last column correctly.

Over the rest of these notes we build up the tools in this order:

1. **Statements:** things that are true or false.
2. **Connectives:** how to combine statements (NOT, AND, OR, IF-THEN, IFF).
3. **Truth tables:** how to compute the truth value of a combination.
4. **De Morgan's laws:** how to negate a compound condition.
5. **The contrapositive:** which rearrangements of “if-then” keep its meaning.

2 Statements

2.1 What Is a Statement?

Logic does not work with every kind of sentence. It works only with sentences that make a claim about the world, so that the world can either agree or disagree with them.

Definition: Statement

A **statement** (also called a *proposition*) is a sentence that

- describes a **state of affairs**,
- **alleges a fact**, that is, it makes a claim,
- and therefore carries a **truth value**: TRUE or FALSE, never both.

A quick test for whether a sentence is a statement: could you sensibly reply “that’s true” or “that’s false”? Compare these two lists.

These ARE statements	These are NOT
“I washed the car.”	“What time is it?”
“ $x = 1$ ”	“ <i>Venga aquí!</i> ” (“Come here!”)
“TN is a state.”	“Blue.”

Each sentence on the left makes a claim that is either correct or not. A question asks for information rather than claiming anything, a command asks for an action, and a single word like “Blue” names something without asserting anything about it, so none of the right-hand examples has a truth value.

Worked example: sorting sentences from a clinic

Apply the test to each sentence below, the kind you might find in a chart, a study protocol, or a hallway conversation. For each one, ask whether “that’s true” or “that’s false” would be a sensible reply.

Sentence	Statement?	Why
“The patient’s systolic blood pressure is 150 mmHg.”	Yes	A claim that the chart can confirm or refute.
“Nashville is the capital of Kentucky.”	Yes	It is false , but it is still a claim, so it is still a statement.
“Please record the patient’s weight.”	No	A command: it asks for an action and claims nothing.
“Has the patient fasted since midnight?”	No	A question: it asks for a truth value instead of stating one.
“Hemoglobin A1c.”	No	The name of a lab test; nothing is claimed about it.
“Every participant in the trial is at least 18 years old.”	Yes	One claim about the whole group, which is either correct or not.
“The patient in room 4 has type 2 diabetes.”	Yes	It has a truth value even if the lab results are not back yet.
“Good morning!”	No	A greeting: there is nothing to agree or disagree with.

Watch out: “statement” does not mean “true statement”

A false sentence is still a statement: “Nashville is the capital of Kentucky” has a truth value, and that value is F. Likewise, a statement whose truth value we do not yet *know* is still a statement. Being a statement is about *having* exactly one of the two values; it is not about the value being T, and not about anyone knowing which one it is.

The sentence “ $x = 1$ ” in the first table deserves a second look. Whether it is true depends on which number x stands for, but once x has a value it is definitely one or the other: true when x is 1, false otherwise. Data science is full of statements like this. “The patient’s A1c is at least 6.5%” is a single sentence, but it is true for some patients and false for others. Applied to a table of patients, it produces one truth value per row, exactly like the filter in Section 1.

Watch out: a claim can be too vague to use

“The patient is elderly” is a claim, but two clinicians may disagree about a 66-year-old, and a computer cannot evaluate it at all. Before a sentence can serve as a condition in a study or a query, replace each vague word with a precise test: “The patient is at least 65 years old.” The precise version has a truth value that everyone computes the same way, which was the whole point of Section 1.

2.2 Writing the Two Truth Values

Every statement carries one of exactly two values. Different fields write those two values differently, but the choice is one of **notation**, never of meaning:

Convention	“yes”	“no”
written out	TRUE	FALSE
abbreviated	T	F
as bits (computing)	1	0
as spin (quantum)	up $\uparrow$	down $\downarrow$

What actually matters is that there are exactly **two** values; that is all the logic depends on. Everything we prove will hold no matter which set of labels you use. These notes use T and F.

You will meet the third row of that table constantly in data work. Python writes the two values as `True` and `False`, and a data set usually stores a yes/no variable, such as “has diabetes,” as a column of 1s and 0s.

Worked example: counting with 1 and 0

Five patients answer the survey question “Do you currently smoke?” with Yes, No, Yes, Yes, No.

1. Each answer is the truth value of a statement about that patient, “this patient currently smokes,” so we record T, F, T, T, F.
2. In the bit convention the same five values are 1, 0, 1, 1, 0.
3. Add them up, and then divide by the number of patients:

$$1 + 0 + 1 + 1 + 0 = 3, \quad \frac{3}{5} = 0.6.$$

The sum, 3, is the number of smokers, and 0.6 is the fraction of patients who smoke. The labels are only notation, but choosing 1 and 0 has a practical payoff: **adding up a column of truth values counts the T rows, and averaging it gives the fraction that are T**. You will use this trick constantly.

Watch out: check how your data set codes yes and no

Not every data set uses 1 and 0. Some surveys code yes as 1 and no as 2, and real records often have a blank where the answer is missing. Our logic has exactly two values, so before you filter or count, decide how each code maps to T or F, and decide what a blank should mean. Summing a column coded 1 and 2 adds up codes, not patients: for the five answers above it would give $1 + 2 + 1 + 1 + 2 = 7$, which counts nothing.

2.3 Naming Statements with Letters

Just as algebra uses x to stand for a number, logic uses letters to stand for whole statements, usually p , q , and r :

p = “I washed the car”

q = “I left my house”

Then, instead of writing out “the statement ‘I washed the car’ is true,” we simply write

$$p = \text{T}.$$

Why bother? Because letters let us study the *structure* of an argument without being distracted by what it is about. Logic should work regardless of what the statements actually say, and writing them as letters makes that visible.

It helps to choose letters that remind you of the statement. For the study criterion these notes write d for “the patient is diabetic,” h for “the patient is hypertensive,” o for “the patient is obese,” and g for “the patient is pregnant” (for *gravid*, the clinical word, since p is already our all-purpose letter).

Worked example: finding the basic statements in a rule

Take the rule “The patient is at least 18, has signed the consent form, and is not on dialysis.”

1. **Find the separate claims.** Look for the pieces joined by words like “and,” “or,” and “not.” There are three: one about age, one about consent, one about dialysis.

2. **Name each claim in its positive form:**

a = “The patient is at least 18 years old”

c = “The patient has signed the consent form”

k = “The patient is on dialysis”

3. **Notice what is left over:** the words “and” and “not.” They are not statements; they are the glue that Section 3 turns into symbols.

Why name “is on dialysis” rather than “is not on dialysis”? Other rules will mention the same fact in its positive form (“patients on dialysis need a daily potassium check”), and we want one letter for one fact. The “not” is then handled by a connective, which is exactly what connectives are for.

Worked example: a rule about lab values

Take “Either the fasting glucose is at least 126 mg/dL or the A1c is at least 6.5%.” The two claims are

f = “The fasting glucose is at least 126 mg/dL”

a = “The A1c is at least 6.5%”

and the sentence becomes “either f or a .” Once the letters are in place, the sentence is reduced to its skeleton, and the skeleton is all the logic needs.

2.4 Two Different Equals Signs?

A fair objection is that these two lines cannot both be using “=” in the same way:

p = “I washed the car”

p = T

They are doing different jobs.

- The first line **names**: it says “let p stand for this statement.” Some books write p : instead, to keep the two uses apart.
- The second line **evaluates**: it reports the one thing about p that the logic uses, its truth value.

Algebra already works this way. Let x count the patients in a clinic. Once you write $x = 5$, the arithmetic no longer cares what was counted: $5 + 3 = 8$ regardless. Logic does the same thing, with only **two** possible values instead of infinitely many.

The analogy goes one step further. In algebra the same letter can take different values in different problems; in a data set the same statement takes different truth values for different patients. Name $d =$ “the patient is diabetic” once, and then evaluate it patient by patient:

Patient	P1	P2	P3	P4
Value of d	T	T	F	T

So $d = T$ for patient P1 and $d = F$ for patient P3. The naming line never changes; the evaluating line changes from row to row. (These are four of the eight patients used in the notebook for this unit; all eight return in Sections 4 and 5.)

3 Connectives and Truth Tables

3.1 Combining Statements

Boolean algebra was developed by George Boole (1815 to 1864). Ordinary algebra combines *numbers* with $+$ and $\times$; Boolean algebra combines *statements* with NOT, AND, and OR. The symbols that combine statements are called **connectives**:

Symbol	Name	Read as
$\neg p$	negation	“not p ”
$p \wedge q$	conjunction	“ p and q ”
$p \vee q$	disjunction	“ p or q ”
$p \rightarrow q$	implication	“if p , then q ”
$p \leftrightarrow q$	bi-implication	“ p if and only if q ”

A connective takes one or two statements and produces a new statement whose truth value depends *only* on the truth values of the pieces. That is what makes logic mechanical: to know whether “ p and q ” is true, you never need to know what p and q are about, only whether each one is true.

In English, each connective hides behind several different words. These are the usual signals:

Connective	English words that usually signal it	Example
$\neg$	not, no, non-, without, it is not the case that	“a non -smoker”
$\wedge$	and, but, also, although, both ... and	“diabetic but not pregnant”
$\vee$	or, either ... or, at least one of	“a smoker or a former smoker”
$\rightarrow$	if ... then, whenever, implies	“ if pregnant, then excluded”
$\leftrightarrow$	if and only if, exactly when	“flagged exactly when potassium is above 5.5”

These are signals, not guarantees. English is flexible, and later in this section we will meet an “and” that the logic has to read as OR. Always check the meaning, not just the word.

3.2 The Truth Table

Definition: Truth table

A **truth table** lists **every** possible combination of truth values for the inputs, and gives the resulting truth value of the output. It is the complete definition of a connective: nothing is left to interpretation.

To read one: each **row** is one possible assignment of truth values to the inputs, and each **column** is one statement (an input, an intermediate piece, or the final result). Every connective in the table above is defined by exactly one such truth table, and we build them next.

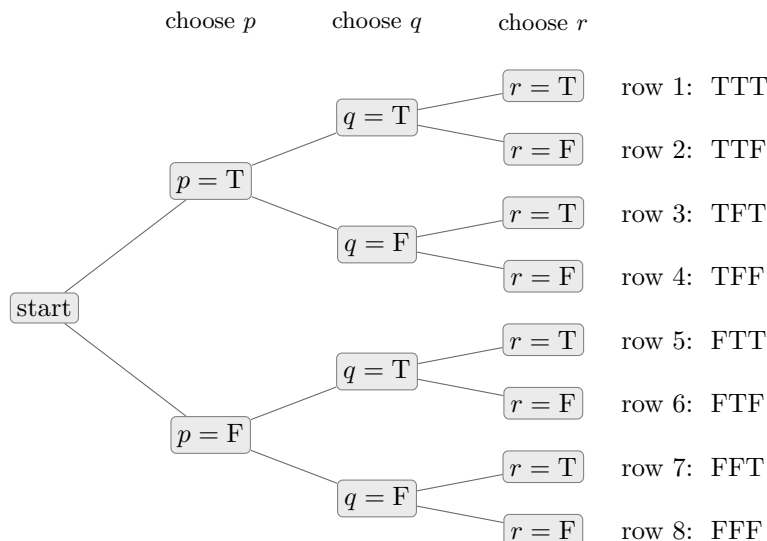
How many rows, and in what order

A truth table must list *every* combination of input values, so its size depends only on how many inputs there are. Each input has two possible values, and each new input doubles the count:

$$\underbrace{2}_{1 \text{ input}}, \quad \underbrace{2 \times 2 = 4}_{2 \text{ inputs}}, \quad \underbrace{2 \times 2 \times 2 = 8}_{3 \text{ inputs}}, \quad \underbrace{2 \times 2 \times 2 \times 2 = 16}_{4 \text{ inputs}}.$$

The full study criterion, with its four statements d , h , o , g , would need a 16-row table.

We also list the rows in a fixed order, so that two tables can be compared row by row. The order used throughout these notes: the first input is T in the top half of the rows and F in the bottom half; the next input alternates in blocks half as long; the last input alternates every row. The tree below shows why this lists every combination exactly once. Each level makes one T/F choice for one input, and each path from left to right is one row.



The top path chooses T three times and gives row 1, TTT; the bottom path gives row 8, FFF. Stop the tree after the q level and you get the four rows TT, TF, FT, FF that every two-input table below uses, in that order.

3.3 Negation (NOT)

The **negation** of p is written $\neg p$ (or NOT p). It flips the truth value.

p = “I went to the store”
 $\neg p$ = “I did NOT go to the store”

p	$\neg p$
T	F
F	T

Negating twice returns you to where you started: $\neg(\neg p) = p$.

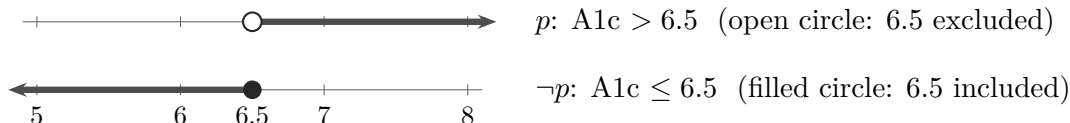
Intuitively, $\neg p$ is the claim that p is false. So the negation must be true in exactly the situations where p is false: no situation may be left out, and none may be counted on both sides.

Worked example: negating a lab-value statement

Let p = “The patient’s A1c is above 6.5%.” Its negation is “The patient’s A1c is *not* above 6.5%,” which in plain terms says “The patient’s A1c is **at most** 6.5%.” Check three patients:

A1c (%)	p : A1c > 6.5	$\neg p$: A1c ≤ 6.5
7.0	T	F
6.5	F	T
6.0	F	T

The middle row is the one people get wrong. An A1c of exactly 6.5 is not *above* 6.5, so p is false there and $\neg p$ must be true. That is why the negation is “at most” ($\leq$) and not “below” ($<$): writing “below 6.5” would leave the patient at exactly 6.5 in neither group. The picture shows the two statements splitting the number line with nothing left over.



The same care applies to every comparison. Each one has an exact negation, and the boundary always switches sides:

Statement	Its negation
$x > a$	$x \leq a$
$x \geq a$	$x < a$
$x < a$	$x \geq a$
$x \leq a$	$x > a$
$x = a$	$x \neq a$

Watch out: negation is not the opposite

The negation of “the patient is hypertensive” is “the patient is *not* hypertensive,” which includes every patient with normal blood pressure. It is not “the patient is hypotensive,” which is a much stronger claim about *low* blood pressure. Likewise, the negation of “the test was positive” is “the test was not positive,” which covers negative results and, in real data, inconclusive ones too. To negate, simply deny the claim; do not jump to its opposite extreme.

Worked example: double negatives in English

Consider “It is not true that the patient did not fast.” Let f = “The patient fasted.” The inner phrase “did not fast” is $\neg f$, and the outer “it is not true that” negates it again:

$$\neg(\neg f) = f.$$

So the sentence says, simply, that the patient fasted. In the same way, “The patient is not a non-smoker” means the patient smokes. Double negatives are easy to misread in a protocol; rewriting them with $\neg(\neg p) = p$ removes the confusion.

3.4 Conjunction (AND)

The **conjunction** of p and q is written $p \wedge q$, read “ p and q .”

p = “I went to the store”

q = “I left my house”

$p \wedge q$ = “I went to the store AND I left my house”

p	q	$p \wedge q$
T	T	T
T	F	F
F	T	F
F	F	F

$p \wedge q$ is TRUE only when **both** inputs are TRUE. If either part of the claim fails, the whole claim fails. An AND makes two claims at once, so it has two ways to go wrong and only one way to be right.

Worked example: an age range is an AND

A study enrolls patients “aged 18 to 65 inclusive.” That single phrase is really two conditions that must both hold:

$$(\text{age} \geq 18) \wedge (\text{age} \leq 65).$$

Evaluate it for four patients, one column per piece:

Age	$\text{age} \geq 18$	$\text{age} \leq 65$	both
17	F	T	F
40	T	T	T
65	T	T	T
70	T	F	F

The 17-year-old fails the first condition and the 70-year-old fails the second; either failure is enough to be excluded. The 65-year-old is enrolled, because “inclusive” puts the boundary inside, which the $\leq$ records.

Worked example: “but” is AND

“Diabetic but not pregnant” uses “but” for a contrast in tone, yet logically it only says that both parts hold: $d \wedge \neg g$. Evaluate it for three of the notebook’s patients.

Patient	d	g	$\neg g$	$d \wedge \neg g$
P1	T	F	T	T
P3	F	T	F	F
P7	F	F	T	F

P1 passes both parts. P3 fails both, and P7 fails only the diabetes part, but one failure is all it takes. Notice the order of work: first compute the piece $\neg g$, then combine.

A useful consequence for data work: **adding an AND condition to a filter can only remove rows, never add them**, because a row now has to pass the old conditions *and* the new one.

3.5 Disjunction (OR)

The **disjunction** of p and q is written $p \vee q$, read “ p or q .”

p = “I went to the store”

q = “I left my house”

$p \vee q$ = “I went to the store OR I left my house”

p	q	$p \vee q$
T	T	T
T	F	T
F	T	T
F	F	F

$p \vee q$ is FALSE only when **both** inputs are FALSE. One true part is enough to make the whole claim true.

Worked example: two diagnostic thresholds

Suppose a study counts a patient as having diabetes if the fasting plasma glucose (FPG) is at least 126 mg/dL **or** the A1c is at least 6.5%, two of the standard diagnostic thresholds. Let f and a be those two statements.

Patient	FPG (mg/dL)	A1c (%)	f	a	$f \vee a$
1	110	6.8	F	T	T
2	131	6.1	T	F	T
3	140	7.2	T	T	T
4	98	5.4	F	F	F

Patients 1 and 2 each meet one threshold, which is enough. Patient 3 meets both, and that still counts: one or both will do. Only patient 4, who meets neither, is not counted. Reading the table this way is exactly the one-liner for OR: false only when both parts are false.

Watch out: an English “and” that means OR

A study recruiting “patients under 18 and over 65” wants two groups, the children and the older adults. For a single patient, the condition is

$$(\text{age} < 18) \vee (\text{age} > 65).$$

Typing the English word into a query as AND asks for patients who are under 18 *and at the same time* over 65, which returns nobody. When English lists two *groups* joined by “and,” the condition on one *person* is usually an OR.

The mirror image of the AND remark above: **adding an OR alternative to a filter can only let more rows through**, because a row now passes if it meets the old condition *or* the new one.

3.6 Inclusive vs. Exclusive OR

Look again at the first row of the OR table: when **both** inputs are true, $p \vee q$ is TRUE. That is a deliberate choice, and it is not how we always speak. English uses “or” in two different ways.

- **Inclusive OR** ($p \vee q$) is true if *at least* one of p, q is true. “Applicants with a nursing degree or clinical experience”: having both is fine.
- **Exclusive OR** (XOR) is true if *exactly* one of p, q is true. “Soup or salad”: pick one, not both.

p	q	$p \vee q$ (inclusive)	$p \text{ XOR } q$ (exclusive)
T	T	T	F
T	F	T	T
F	T	T	T
F	F	F	F

The two differ only in the first row. **In mathematics, “or” always means the inclusive one** unless we say otherwise.

Worked example: four applicants

A hiring rule asks for “a nursing degree or clinical experience.” Let n = “has a nursing degree” and e = “has clinical experience.”

Applicant	n	e	$n \vee e$ (inclusive)	$n \text{ XOR } e$ (exclusive)
A (degree and experience)	T	T	T	F
B (degree only)	T	F	T	T
C (experience only)	F	T	T	T
D (neither)	F	F	F	F

Under the inclusive reading, applicant A, the strongest candidate, qualifies. Under the exclusive reading A would be rejected for being *too* qualified, which is obviously not what the employer meant. That is the typical situation: in rules and criteria, “or” is almost always inclusive, which is one reason mathematics makes inclusive the default.

Worked example: when the difference disappears

“The rapid test came back positive or negative.” A single result cannot be both, so the first row of the table (both true) never occurs. Inclusive and exclusive OR differ *only* in that row, so here they agree on every case that can actually happen, and it makes no difference which one you use. The distinction matters only when both parts can be true at the same time.

If you do mean the exclusive version, say so explicitly: “either ... or ..., but not both.” Exercise 4.5 shows how to build XOR out of the connectives we already have.

3.7 Implication (IF-THEN)

Implication is written $p \rightarrow q$, read “if p , then q ” or “ p implies q .” The two parts have names:

- p is the **antecedent** (the hypothesis),
- q is the **consequent** (the conclusion).

p = “I went to the store”

q = “I left my house”

$p \rightarrow q$ = “IF I went to the store, THEN I left my house”

A helpful way to think of an implication is as a **promise**. The only way to break the promise is for p to happen and q to fail. That idea gives the whole truth table:

p	q	$p \rightarrow q$
T	T	T
T	F	F
F	T	T
F	F	T

$p \rightarrow q$ is FALSE only when the antecedent is TRUE and the consequent is FALSE, that is, only when the promise turns out to be a **lie**.

The surprising rows are the last two. If p is FALSE, the implication is TRUE no matter what q is, because the promise was never tested. Take “If I went to the store, then I left my house.” If I never went to the store, I have not broken my word, whether or not I left the house. An implication that is true because its antecedent is false is called **vacuously true**.

English has several ways of saying $p \rightarrow q$, and they all mean the same thing:

- “if p , then q ”
- “ q if p ” (the if-clause is still the antecedent, even when it comes second)
- “ q whenever p ”
- “ p implies q ”

The antecedent is always the part attached to “if” or “whenever,” wherever it sits in the sentence.

Worked example: auditing a clinical rule

Suppose a hospital’s rule is: “If a patient’s potassium is above 5.5 mmol/L, then the chart is flagged.” Let p = “potassium is above 5.5” and q = “the chart is flagged.” Here are five records.

Record	Potassium	Flagged?	p	q	$p \rightarrow q$
A	6.1	yes	T	T	T
B	4.2	no	F	F	T
C	5.9	no	T	F	F
D	4.8	yes	F	T	T
E	5.5	no	F	F	T

1. **Evaluate p for each record.** Watch the boundary: a potassium of exactly 5.5 is not *above* 5.5, so p is F for record E.
2. **Read q from the chart.**
3. **Apply the implication table.** Only one record has p true and q false: record C.

The rule was broken exactly once, by record C. Record D was flagged even though its potassium was normal, perhaps for some other reason, and the rule does not forbid that: it only makes a promise about patients with high potassium. This is the general recipe: **to audit an if-then rule, look only for rows where the antecedent is true and the consequent is false.**

Worked example: vacuous truth in a data set

Take the rule “If a patient is pregnant, then a prenatal visit is recorded.” For a patient who is not pregnant, the antecedent is false, so the rule is vacuously true for that patient, whatever the chart shows.

This matters when compliance is reported as a percentage. In a clinic of 200 patients of whom 10 are pregnant, the rule is automatically satisfied by the other 190. If 4 of the 10 pregnant patients have no prenatal visit recorded, the rule still holds for 196 of the 200 patients,

$$\frac{196}{200} = 98\%,$$

even though it failed for 4 of the 10 patients it is actually about, 40%. A compliance rate computed over everyone is swollen by vacuous passes; the informative rate is computed over the rows where the antecedent is true.

Worked example: an implication about numbers

Consider “If $x > 5$, then $x > 3$.” Check three values of x :

x	$p: x > 5$	$q: x > 3$	$p \rightarrow q$
7	T	T	T
4	F	T	T
1	F	F	T

No value of x can make the antecedent true and the consequent false, because any number above 5 is automatically above 3. So this implication is true for *every* x . For $x = 4$ and $x = 1$ it is true only vacuously. We will reuse this example in Section 6.

Watch out: IF-THEN does not mean “because”

A true implication does not claim that p *causes* q , or that p happens first. “If a patient is diabetic, then an A1c result is on file” can be a perfectly true statement about a clinic’s records, yet diabetes does not cause lab paperwork. And a true implication says nothing about the cases where p is false: from “if p then q ” together with “ p is false,” you can conclude nothing at all about q .

3.8 Bi-implication (IF AND ONLY IF)

Bi-implication is written $p \leftrightarrow q$, read “ p if and only if q ” (often shortened to “ p iff q ”). The implication runs **both ways**:

$$p \leftrightarrow q = (p \rightarrow q) \wedge (q \rightarrow p).$$

We can build its truth table from the two implications:

p	q	$p \rightarrow q$	$q \rightarrow p$	$p \leftrightarrow q$
T	T	T	T	T
T	F	F	T	F
F	T	T	F	F
F	F	T	T	T

The last column is the **AND** of the two middle ones: it is true only where *both* implications hold. So $p \leftrightarrow q$ is TRUE exactly when p and q **match**.

This is the connective behind every mathematical **definition**. When we say a whole number is “even *if and only if* it is divisible by 2,” we mean the two conditions always go together: every even number is divisible by 2, and every number divisible by 2 is even.

Where the name comes from

The two halves of “if and only if” are two different implications.

- “ p **if** q ” puts q in the if-clause, so it means $q \rightarrow p$.
- “ p **only if** q ” says p cannot happen without q , so it means $p \rightarrow q$.

Joining them with “and” gives $(p \rightarrow q) \wedge (q \rightarrow p)$, which is exactly $p \leftrightarrow q$.

“Only if” on its own is common in protocols and easy to misread. “A patient may enroll only if they have signed the consent form” means $\text{enroll} \rightarrow \text{consent}$. Signing is required, but it does not guarantee enrollment: a patient who signs and then turns out to be pregnant is still turned away. (You may hear this put as: consent is *necessary* for enrollment, but not *sufficient*.) The “only if” direction alone does not run backwards.

Worked example: auditing a definition

Suppose a study *defines* a patient as hypertensive if and only if their systolic pressure is at least 130 mmHg, and every record carries a “hypertensive” label. Let p = “the record is labeled hypertensive” and q = “the systolic pressure is at least 130.”

Record	Systolic	Label	p	q	$p \leftrightarrow q$
1	142	yes	T	T	T
2	118	no	F	F	T
3	135	no	F	T	F
4	121	yes	T	F	F
5	130	yes	T	T	T

Records 3 and 4 are mislabeled, in opposite directions: record 3 is missing a label it should have, and record 4 carries a label it should not. A one-way rule would catch only one of these errors (“label $\rightarrow$ systolic at least 130” catches record 4, the reverse catches record 3). The IFF catches both, because it demands that label and measurement **match**. Record 5 sits on the boundary and is labeled correctly, since “at least 130” includes 130.

Worked example: an IFF that fails

Is “ $x^2 = 9$ if and only if $x = 3$ ” true for every number x ? Check each direction.

- “If $x = 3$, then $x^2 = 9$ ”: true, since $3^2 = 9$.
- “If $x^2 = 9$, then $x = 3$ ”: try $x = -3$. Then $x^2 = 9$ is true but $x = 3$ is false, so this direction breaks.

x	$x^2 = 9$	$x = 3$	$x^2 = 9 \leftrightarrow x = 3$
3	T	T	T
-3	T	F	F
0	F	F	T

One direction holding is not enough; an IFF needs both. The correct statement is “ $x^2 = 9$ if and only if $x = 3$ or $x = -3$.”

A last way to think about it: $p \leftrightarrow q$ behaves like an equals sign for truth values. It is true when p and q have the *same* truth value, and it does not claim that they mean the same thing. “ $x = 3$ ” and “ $2x = 6$ ” are different sentences, but they are true for exactly the same values of x , so “ $x = 3 \leftrightarrow 2x = 6$ ” is true for every x .

3.9 How Many Inputs? Unary and Binary

That is every connective we need. Looking back over them, notice **how many statements each one takes**, a property called its **arity**.

Unary vs. binary

- A **unary** operator takes **one** input. Only $\neg$ is unary.
- A **binary** operator takes **two** inputs. $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$, and XOR are all binary.

Arithmetic has the same split. The negative sign in $-x$ is unary: one number in, one number out. The operations $+$, $-$, $\times$, $\div$ are binary: two numbers in, one out.

The arity also fixes the size of each connective’s truth table: a unary connective needs 2 rows, and a binary one needs $2 \times 2 = 4$. That is why the table for $\neg$ above has two rows and all the others have four.

This will matter shortly. If $\wedge$ only ever joins **two** statements, what could $p \wedge q \wedge r$ possibly mean? Section 4 answers that.

3.10 Summary of the Connectives

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \rightarrow q$	$p \leftrightarrow q$	XOR
T	T	F	T	T	T	T	F
T	F	F	F	T	F	F	T
F	T	T	F	T	T	F	T
F	F	T	F	F	T	T	F

Worth memorizing as one-liners

- AND: true only when **both** are true.
- OR: false only when **both** are false.
- IF-THEN: false only when **true** implies **false**.
- IFF: true only when they **match**.
- XOR: true only when they **differ**.

The same information can be drawn as pictures. In each grid below, p runs down the side and q across the top, and the shaded squares are the inputs that make the connective TRUE.

	$q=T$	$q=F$		$q=T$	$q=F$		$q=T$	$q=F$		$q=T$	$q=F$		$q=T$	$q=F$		$q=T$	$q=F$
$p=T$	T	F		T	T		T	F		T	F		T	F		F	T
$p=F$	F	F		T	F		T	T		F	T		T	F		T	F
	$p \wedge q$			$p \vee q$			$p \rightarrow q$			$p \leftrightarrow q$			$p \text{ XOR } q$				

Each picture is one of the one-liners. AND shades a single square (both true). OR leaves a single square blank (both false). IF-THEN leaves blank only the square where p is true and q is false. IFF shades the diagonal where p and q match, and XOR shades the other diagonal, where they differ.

The same connectives in software

Every data tool you will use has these connectives built in, under slightly different spellings:

Logic	Python	pandas / NumPy	SQL
$\neg p$	not p	~p	NOT p
$p \wedge q$	p and q	p & q	p AND q
$p \vee q$	p or q	p q	p OR q

For truth values, Python's `==` acts as IFF and `!=` acts as XOR, since each simply tests whether the two values match. There is no IF-THEN operator in any of these tools; Section 4 shows how to write one using the others.

3.11 Extra Practice

Exercise 3.1

Let $p = "7 > 3"$ and $q = "2 + 2 = 5."$ Find the truth value of each of the following:

$$\neg q, \quad p \wedge q, \quad p \vee q, \quad p \rightarrow q, \quad q \rightarrow p, \quad p \leftrightarrow q, \quad p \text{ XOR } q.$$

Solution to Exercise 3.1

First evaluate the two inputs: $7 > 3$ is true and $2 + 2 = 5$ is false, so $p = \text{T}$ and $q = \text{F}$. Every answer can now be read from the second row ($p = \text{T}$, $q = \text{F}$) of the summary table.

- $\neg q = \neg \text{F} = \text{T}$.
- $p \wedge q = \text{T} \wedge \text{F} = \text{F}$: AND needs both.
- $p \vee q = \text{T} \vee \text{F} = \text{T}$: one true part is enough.
- $p \rightarrow q = \text{T} \rightarrow \text{F} = \text{F}$: a true antecedent with a false consequent, the one way to break the promise.
- $q \rightarrow p = \text{F} \rightarrow \text{T} = \text{T}$: vacuously true, since the antecedent q is false.
- $p \leftrightarrow q = \text{F}$: the values do not match.
- $p \text{ XOR } q = \text{T}$: the values differ.

Notice that $p \rightarrow q$ and $q \rightarrow p$ came out different. For implication, the order of the two parts matters; Section 4 returns to this.

Exercise 3.2

Write the exact negation of each statement in plain English, taking care at the boundary.

- "The patient's temperature is above 38.0°C ."
- "The patient is at least 18 years old."
- "The patient's BMI equals 30."
- "The patient is not a smoker."

Solution to Exercise 3.2

- "Above" is $>$, so the negation is $\leq$: "The patient's temperature is **at most** 38.0°C ." A temperature of exactly 38.0 belongs to the negation.
- "At least" is $\geq$, so the negation is $<$: "The patient is **younger than** 18." Here the boundary age 18 belongs to the original statement, not the negation.
- The negation of $=$ is $\neq$: "The patient's BMI is **not** 30," which covers values both below and above 30.
- This statement is already a negation, $\neg s$ with $s =$ "the patient is a smoker." Its negation is $\neg(\neg s) = s$: "The patient **is** a smoker."

In (a) to (c) the boundary value always ends up on exactly one side, so every patient is covered

by exactly one of the statement and its negation.

Exercise 3.3

A clinic's rule reads: "If a patient is prescribed warfarin, then an INR test is recorded." Let w = "prescribed warfarin" and i = "INR test recorded." For each record below, decide whether the rule holds, and say which records satisfy it only vacuously.

Record	On warfarin?	INR recorded?
1	yes	yes
2	yes	no
3	no	yes
4	no	no

Solution to Exercise 3.3

Translate each record into truth values and apply the implication table.

Record	w	i	$w \rightarrow i$	Comment
1	T	T	T	rule tested and kept
2	T	F	F	violation
3	F	T	T	vacuous
4	F	F	T	vacuous

Only record 2 breaks the rule: a patient on warfarin with no INR on file. Records 3 and 4 are not on warfarin, so the rule makes no promise about them and holds vacuously, whether or not an INR was recorded. Record 1 is the only one where the rule was actually put to the test and kept.

Exercise 3.4

"A patient may be discharged only if their fever has resolved." Let d = "the patient is discharged" and f = "the fever has resolved."

- Write the rule as an implication.
- A patient's fever has resolved. Does the rule require that the patient be discharged?
- A patient was discharged, and the rule was followed. What do you know about their fever?

Solution to Exercise 3.4

- " p only if q " means $p \rightarrow q$, so the rule is $d \rightarrow f$: if the patient is discharged, then the fever has resolved.
- No. Knowing $f = T$ settles nothing about d : both rows with $f = T$ (discharged or not) make $d \rightarrow f$ true. A resolved fever is required for discharge, but other things may keep the patient in hospital.
- The fever has resolved. With $d = T$, the only row that would break the rule is $f = F$,

and the rule was followed, so $f = T$.

The rule constrains discharged patients only; it runs from d to f and not back.

4 Compound Statements

4.1 Building Bigger Expressions

Connectives can be nested, exactly like arithmetic operations, and parentheses decide the order in which they apply:

$$\neg(p \wedge q) \quad \text{is not the same as} \quad (\neg p) \wedge q.$$

The first negates the whole conjunction; the second negates only p and then joins the result to q .

To see the difference concretely, let p = “the patient is diabetic” and q = “the patient is hypertensive,” and take patient P2, who is diabetic but not hypertensive, so $p = T$ and $q = F$. Work each expression from the inside out:

$$\begin{aligned}\neg(p \wedge q) &= \neg(T \wedge F) = \neg F = T, \\ (\neg p) \wedge q &= (\neg T) \wedge F = F \wedge F = F.\end{aligned}$$

The first says “not both diabetic and hypertensive,” which is true for P2, who has only one of the two. The second says “not diabetic, and hypertensive,” which is false for P2 on both counts. Same letters, same symbols, different parentheses, different answers.

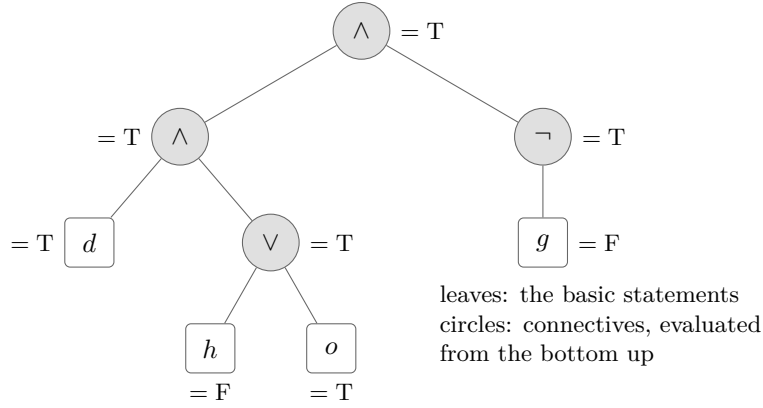
Strategy for a compound truth table

1. Write one column for each input statement, and give every combination of their truth values its own row.
2. Add a column for each **intermediate** piece, working from the inside of the parentheses out.
3. The last column is the answer.

Doing this by hand a few times is how the rules stop feeling arbitrary. After that, let a computer do it: see the notebook `U1-1_Logic-1_ConnectivesAndTruthTables.ipynb`.

Seeing the structure as a tree

Every compound statement has a structure you can draw. The outermost connective sits at the top, the pieces it joins hang below it, and the basic statements sit at the bottom. Here is the study criterion $d \wedge (h \vee o) \wedge \neg g$, grouped as $(d \wedge (h \vee o)) \wedge \neg g$, with the truth values for patient P2 (diabetic, not hypertensive, obese, not pregnant) written beside each piece.



“Working from the inside out” means climbing the tree from the leaves:

$$\begin{aligned}
 h \vee o &= F \vee T = T, \\
 d \wedge (h \vee o) &= T \wedge T = T, \\
 \neg g &= \neg F = T, \\
 (d \wedge (h \vee o)) \wedge \neg g &= T \wedge T = T.
 \end{aligned}$$

Patient P2 is eligible. The parentheses in a formula are nothing more than a way of writing this tree on one line.

Worked example: the same criterion for patient P3

Patient P3 is not diabetic, is hypertensive, is obese, and is pregnant: $d = F$, $h = T$, $o = T$, $g = T$. Climb the same tree:

$$\begin{aligned}
 h \vee o &= T \vee T = T, \\
 d \wedge (h \vee o) &= F \wedge T = F, \\
 \neg g &= \neg T = F, \\
 (d \wedge (h \vee o)) \wedge \neg g &= F \wedge F = F.
 \end{aligned}$$

P3 is not eligible, and for two separate reasons: no diabetes, and pregnant. Either reason alone would have been enough, because an AND fails as soon as one of its pieces fails.

Worked example: a full table with three inputs

A clinic flags a patient for cardiology review if they report chest pain or shortness of breath, and have *not* had an ECG in the last month. Let

$$p = \text{“chest pain”}, \quad q = \text{“shortness of breath”}, \quad r = \text{“ECG in the last month”},$$

so the flag is $(p \vee q) \wedge \neg r$.

1. Three inputs give $2 \times 2 \times 2 = 8$ rows, listed in the standard order.
2. The intermediate pieces are $p \vee q$ and $\neg r$; compute each one.

3. AND the two intermediate columns together.

p	q	r	$p \vee q$	$\neg r$	$(p \vee q) \wedge \neg r$
T	T	T	T	F	F
T	T	F	T	T	T
T	F	T	T	F	F
T	F	F	T	T	T
F	T	T	T	F	F
F	T	F	T	T	T
F	F	T	F	F	F
F	F	F	F	T	F

The final column is T in rows 2, 4, and 6: exactly the patients with at least one symptom and no recent ECG. Rows 1, 3, 5, and 7 are not flagged because a recent ECG makes $\neg r$ false, and row 8 is not flagged because a patient with no symptoms fails the OR. Reading the table back in words is the best check that the formula says what the clinic meant.

Exercise 4.1

Let $p = \text{T}$, $q = \text{F}$, and $r = \text{T}$. Evaluate each expression from the inside out.

- (a) $p \wedge \neg q$
- (b) $\neg(p \vee r)$
- (c) $(p \rightarrow q) \vee r$
- (d) $p \rightarrow (q \wedge r)$
- (e) $(p \leftrightarrow q) \vee (q \leftrightarrow r)$
- (f) $\neg p \rightarrow (q \wedge r)$

Solution to Exercise 4.1

Replace each letter by its value, then evaluate the innermost pieces first.

- (a) $\text{T} \wedge \neg \text{F} = \text{T} \wedge \text{T} = \text{T}$.
- (b) $\neg(\text{T} \vee \text{T}) = \neg \text{T} = \text{F}$.
- (c) $(\text{T} \rightarrow \text{F}) \vee \text{T} = \text{F} \vee \text{T} = \text{T}$. The implication fails, but the OR is rescued by r .
- (d) $\text{T} \rightarrow (\text{F} \wedge \text{T}) = \text{T} \rightarrow \text{F} = \text{F}$. A true antecedent and a false consequent.
- (e) $(\text{T} \leftrightarrow \text{F}) \vee (\text{F} \leftrightarrow \text{T}) = \text{F} \vee \text{F} = \text{F}$. Neither pair matches.
- (f) $\neg \text{T} \rightarrow (\text{F} \wedge \text{T}) = \text{F} \rightarrow \text{F} = \text{T}$. Vacuously true: the antecedent $\neg p$ is false.

Compare (d) and (f): both have the consequent $q \wedge r = \text{F}$, yet one is false and the other true. The difference is entirely in the antecedent.

4.2 Chaining ANDs and ORs

Because $\wedge$ is binary, $p \wedge q \wedge r$ must be read as *two* ANDs, one after the other. Both $\wedge$ and $\vee$ are well behaved in two ways that make this harmless.

Commutative: *order* does not matter

$$p \wedge q = q \wedge p \qquad p \vee q = q \vee p$$

Exactly as $a + b = b + a$ and $a \times b = b \times a$.

Associative: *grouping* does not matter

$$(p \wedge q) \wedge r = p \wedge (q \wedge r) \qquad (p \vee q) \vee r = p \vee (q \vee r)$$

Exactly as $(1 + 2) + 3 = 1 + (2 + 3)$, which is why we are allowed to write $1 + 2 + 3$.

Together these let us drop the parentheses entirely. $p \wedge q \wedge r \wedge s$ says “all four are true,” and $p \vee q \vee r \vee s$ says “at least one of the four is,” in whatever order you like.

That reading gives a fast way to evaluate a long chain without building any table:

- an **AND chain** is F as soon as *one* piece is F;
- an **OR chain** is T as soon as *one* piece is T.

Worked example: an inclusion list is an AND chain

A trial enrolls patients who are at least 18 (a), have signed consent (c), are not pregnant ($\neg g$), and have an A1c of at least 6.5% (e). The criterion is the chain $a \wedge c \wedge \neg g \wedge e$. Take a 52-year-old patient who has signed consent, is not pregnant, and has an A1c of 6.2%:

$$a \wedge c \wedge \neg g \wedge e = T \wedge T \wedge T \wedge F = F.$$

One failed condition sinks the whole chain. And because order does not matter, we could have checked the A1c first and stopped right there.

Worked example: an exclusion list is an OR chain

The same trial excludes anyone who is pregnant (g), on dialysis (k), or under 18 (u): the chain $g \vee k \vee u$. Take a 45-year-old patient who is not pregnant but is on dialysis:

$$g \vee k \vee u = F \vee T \vee F = T.$$

The patient is excluded; one reason is enough.

This is why study protocols are written the way they are. **Inclusion criteria form an AND chain and exclusion criteria form an OR chain:** a patient must pass *every* inclusion criterion, but hitting *any one* exclusion criterion is enough to be turned away.

Exercise 4.2

Prove that $\vee$ is associative, $(p \vee q) \vee r = p \vee (q \vee r)$, by building both truth tables.

Solution to Exercise 4.2

Three inputs give 8 rows. For the left side compute $p \vee q$ first and then OR it with r ; for the right side compute $q \vee r$ first and then OR p with it.

p	q	r	$p \vee q$	$(p \vee q) \vee r$	$q \vee r$	$p \vee (q \vee r)$
T	T	T	T	T	T	T
T	T	F	T	T	T	T
T	F	T	T	T	T	T
T	F	F	T	T	F	T
F	T	T	T	T	T	T
F	T	F	T	T	T	T
F	F	T	F	T	T	T
F	F	F	F	F	F	F

The 5th and 7th columns agree on all eight rows, so the two sides are logically equivalent. ■
Both are false only in the last row, where all three inputs are false: whichever way you group it, the expression says “at least one of the three is true.”

4.3 Where the Analogy Stops

Implication fails **both** tests, exactly as subtraction does.

Order matters.

$$p \rightarrow q \neq q \rightarrow p \qquad 8 - 3 \neq 3 - 8$$

Swapping the two sides of an implication gives its **converse**, which is a different statement, as we will see in Section 6.

Grouping matters.

$$(p \rightarrow q) \rightarrow r \neq p \rightarrow (q \rightarrow r)$$

Exactly as $(8 - 3) - 2 = 3$ but $8 - (3 - 2) = 7$.

So

Never write $p \rightarrow q \rightarrow r$. Mixing $\wedge$ with $\vee$ needs parentheses too, as the next two subsections show.

Worked example: one row is enough to show a difference

To show that two statements are *different*, you do not need a whole table; a single row where they disagree is enough.

For order, take the row $p = \text{T}$, $q = \text{F}$: then $p \rightarrow q = \text{F}$ but $q \rightarrow p = \text{T}$. The two also differ in the row $p = \text{F}$, $q = \text{T}$, which is easy to picture with the store example. On a day I left the house to go to work and never went to the store, “If I went to the store, then I left my house” is still true (vacuously), but “If I left my house, then I went to the store” is false.

For grouping, the full table shows exactly where the two versions part ways.

p	q	r	$p \rightarrow q$	$(p \rightarrow q) \rightarrow r$	$q \rightarrow r$	$p \rightarrow (q \rightarrow r)$
T	T	T	T	T	T	T
T	T	F	T	F	F	F
T	F	T	F	T	T	T
T	F	F	F	T	T	T
F	T	T	T	T	T	T
F	T	F	T	F	F	T
F	F	T	T	T	T	T
F	F	F	T	F	T	T

The 5th and 7th columns disagree in rows 6 and 8, the rows where p and r are both false. There, $p \rightarrow (q \rightarrow r)$ is vacuously true because its antecedent p is false. But in $(p \rightarrow q) \rightarrow r$ the antecedent is $p \rightarrow q$, which is vacuously *true* when p is false, and a true antecedent with the false consequent r makes the whole thing false. Written without parentheses, $p \rightarrow q \rightarrow r$ could mean either one, which is why we never write it.

4.4 Why Parentheses Matter

Are $(p \wedge q) \vee r$ and $p \wedge (q \vee r)$ really different? Three statements give $2 \times 2 \times 2 = 8$ combinations of truth values, one row each. We build both columns and compare.

p	q	r	$p \wedge q$	$(p \wedge q) \vee r$	$q \vee r$	$p \wedge (q \vee r)$
T	T	T	T	T	T	T
T	T	F	T	T	T	T
T	F	T	F	T	T	T
T	F	F	F	F	F	F
F	T	T	F	T	T	F
F	T	F	F	F	T	F
F	F	T	F	T	T	F
F	F	F	F	F	F	F

The fifth and last columns disagree on two rows, and both are rows where p is FALSE and r is TRUE. The parentheses decide whether r is allowed to carry the whole statement **by itself**, or must still get past p first.

4.5 What That Difference Means

Read the two groupings aloud, with d = diabetic, h = hypertensive, o = obese.

- $(d \wedge h) \vee o$: “Diabetic **and** hypertensive, **or else** obese.” Being obese is enough on its own; the other two never have to hold.
- $d \wedge (h \vee o)$: “Diabetic, **and** on top of that hypertensive or obese.” Diabetes is mandatory; being obese only helps once you already have it.

A single patient tells them apart. A patient who is **obese but not diabetic** satisfies the first and fails the second: exactly the “ p FALSE, r TRUE” rows from the table. Same three conditions, same three words, one pair of parentheses, and a different cohort. The study criterion from Section 1 was the second grouping, $d \wedge (h \vee o)$.

Worked example: both readings on eight real rows

Here are the eight patients from the notebook, with both groupings evaluated for each.

Patient	d	h	o	$(d \wedge h) \vee o$	$d \wedge (h \vee o)$
P1	T	T	F	T	T
P2	T	F	T	T	T
P3	F	T	T	T	F
P4	T	T	F	T	T
P5	F	F	F	F	F
P6	T	F	T	T	T
P7	F	T	T	T	F
P8	F	F	F	F	F

The first reading selects six patients and the second selects four. The two patients on whom they disagree, P3 and P7, are both obese and not diabetic, exactly as the truth table predicted. On a real database with thousands of rows, that one pair of parentheses can decide whether a large number of patients are enrolled or left out.

Watch out: software picks a grouping for you

If you leave out the parentheses, a computer does not ask which reading you meant; it silently applies its own default. In SQL and in Python, AND is applied before OR, so the query

WHERE diabetic AND hypertensive OR obese

is read as $(d \wedge h) \vee o$, the six-patient reading, which is *not* the study's criterion. That default is a convention of the software, not something logic promises, so do not rely on it: **write the parentheses every time you mix AND with OR.**

4.6 Translating Between English and Symbols

Moving between English and symbols is the practical skill behind everything so far. It follows a short procedure.

Translating a rule into symbols

1. Find the basic statements, and name each one in its positive form.
2. Find the connective words: not, and, but, or, if ... then, only if, if and only if.
3. Decide which pieces belong together, and put parentheses around them.
4. Read the formula back in plain English, and check that it says what the rule meant.

Worked example 1: a single connective

“The patient is at least 65 and has chronic kidney disease.” The basic statements are a = “at least 65” and k = “has chronic kidney disease,” joined by “and”:

$$a \wedge k.$$

Read back: “at least 65, and has chronic kidney disease.” Nothing to group, nothing lost.

Worked example 2: grouping an OR inside an AND

A screening program invites “current or former smokers aged 50 to 80, but not anyone already diagnosed with lung cancer.” Name the pieces:

$$s = \text{“current smoker”}, \quad f = \text{“former smoker”}, \quad c = \text{“diagnosed with lung cancer”}, \\ a = \text{“age } \geq 50\text{”}, \quad b = \text{“age } \leq 80\text{”}.$$

“Current or former smokers” is *one* requirement, so $s \vee f$ must be grouped. The age range is two conditions ANDed together, as we saw in Section 3. “But not” is $\wedge \neg$. Putting it together:

$$(s \vee f) \wedge a \wedge b \wedge \neg c.$$

Read back: “a current or former smoker, and at least 50, and at most 80, and not diagnosed with lung cancer.” That matches the rule. Without the parentheses, a computer applying AND before OR would read $s \vee (f \wedge a \wedge b \wedge \neg c)$ and invite *every* current smoker, whatever their age or diagnosis.

Worked example 3: a policy made of two rules

A clinic’s policy: “Every patient on warfarin must have an INR recorded, and no patient with a prior major bleed may be on warfarin.” Name w = “on warfarin,” i = “INR recorded,” b = “prior major bleed.”

- “Every patient on warfarin must have an INR recorded” is a promise about warfarin patients: $w \rightarrow i$.
- “No patient with a prior major bleed may be on warfarin” says: if a patient had a prior bleed, then they are not on warfarin: $b \rightarrow \neg w$.

The policy asks for both, so it is

$$(w \rightarrow i) \wedge (b \rightarrow \neg w).$$

Now check one patient: on warfarin, INR recorded, and a prior major bleed, so $w = \text{T}$, $i = \text{T}$, $b = \text{T}$:

$$(\text{T} \rightarrow \text{T}) \wedge (\text{T} \rightarrow \neg \text{T}) = \text{T} \wedge (\text{T} \rightarrow \text{F}) = \text{T} \wedge \text{F} = \text{F}.$$

The patient violates the policy: the INR rule is kept, but the bleeding rule is broken.

Worked example 4: symbols back into English

With d = diabetic, h = hypertensive, g = pregnant, and m = “on blood-pressure medication”:

Formula	In English
$d \wedge \neg g$	diabetic but not pregnant
$\neg(d \vee h)$	neither diabetic nor hypertensive
$\neg d \vee \neg h$	not diabetic or not hypertensive, that is, missing at least one of the two conditions
$(d \wedge h) \rightarrow m$	every patient who is both diabetic and hypertensive is on blood-pressure medication

The second and third rows look alike but describe different groups: the second only patients with *neither* condition, the third anyone missing *at least one*. Section 5 explains exactly how they are related.

Watch out: “neither ... nor” denies both

“Neither diabetic nor hypertensive” is $\neg(d \vee h)$: it is true only for patients who have neither condition. A common slip is to write $\neg d \vee \neg h$, which is true for anyone missing at least one condition, a much larger group. Check with a patient who is diabetic but not hypertensive: they are *not* “neither diabetic nor hypertensive,” yet $\neg d \vee \neg h = F \vee T = T$.

Exercise 4.3

Translate each rule into symbols. Use o = obese, h = hypertensive, g = pregnant, u = under 18, s = “a guardian’s signature is on file,” e = eligible, d = diabetic.

- (a) “Patients who are obese or hypertensive, and who are not pregnant.”
- (b) “If a patient is under 18, then a guardian’s signature is on file.”
- (c) “A patient is eligible if and only if they are diabetic and not pregnant.”

Solution to Exercise 4.3

- (a) $(o \vee h) \wedge \neg g$. The parentheses are essential: “obese or hypertensive” is one requirement, and “not pregnant” applies to everyone. Without them, $o \vee h \wedge \neg g$ would (under the AND-first default) let in every obese patient, pregnant or not.
- (b) $u \rightarrow s$. The rule makes a promise only about patients under 18; for adults it is vacuously true.
- (c) $e \leftrightarrow (d \wedge \neg g)$. The phrase “if and only if” says eligibility and the condition must match: every eligible patient is diabetic and not pregnant, and every diabetic, non-pregnant patient is eligible.

4.7 Worked Example: $\neg p \vee q$

Build the truth table for $\neg p \vee q$. The only intermediate piece is $\neg p$, so we compute it first and then OR it with q .

p	q	$\neg p$	$\neg p \vee q$
T	T	F	T
T	F	F	F
F	T	T	T
F	F	T	T

Now compare with the implication table. The final column is T, F, T, T, **identical** to the column for $p \rightarrow q$. So

$$p \rightarrow q = \neg p \vee q.$$

Implication is not really a new idea: it can be built from NOT and OR. In words, “if p then q ” says “either p didn’t happen, or q did.”

This rewrite has a very practical use. Software has no IF-THEN operator for truth values (Section 3), so a rule such as “if pregnant, then excluded” is checked in a query as

NOT pregnant OR excluded

which is $\neg g \vee x$ with $x = \text{“excluded”}$: either the patient is not pregnant, or they were excluded. A patient passes the check exactly when the rule holds for them.

4.8 Logical Equivalence

Definition: Logical equivalence

Two compound statements are **logically equivalent** if their truth tables have the **same final column**, that is, they agree on every possible input.

Equivalent statements can be swapped for one another freely. That is what makes them useful: we can **rewrite** a complicated condition as a simpler one without changing its meaning.

This is also a proof technique. To prove two expressions are equivalent, **compute both truth tables and compare**. A truth table has finitely many rows, so checking them all is a complete proof.

We have already collected several equivalences:

- $\neg(\neg p) = p$ (double negation);
- $p \wedge q = q \wedge p$ and $p \vee q = q \vee p$ (commutative), and the associative laws;
- $p \rightarrow q = \neg p \vee q$;
- $p \leftrightarrow q = (p \rightarrow q) \wedge (q \rightarrow p)$.

And a non-example: $p \rightarrow q$ and $q \rightarrow p$ are *not* equivalent, because they disagree in the row $p = \text{T}$, $q = \text{F}$. Notice the asymmetry: **proving an equivalence needs every row, but disproving one needs only a single row where the two sides disagree**.

Worked example: rewriting a rule step by step

Once some equivalences are proved, we can rewrite expressions one justified step at a time, just as in algebra, without drawing a new table. Take the rule “If a patient is diabetic, then the patient is on metformin or on insulin”: $d \rightarrow (m \vee i)$.

$$\begin{aligned} d \rightarrow (m \vee i) &= \neg d \vee (m \vee i) & (p \rightarrow q = \neg p \vee q, \text{ with } p = d \text{ and } q = m \vee i) \\ &= \neg d \vee m \vee i & (\text{associative, so the parentheses can go}) \end{aligned}$$

In words: a patient satisfies the rule if they are not diabetic, or on metformin, or on insulin. The if-then has become a plain OR chain, which a query can check directly. (Building the 8-row tables for both sides confirms they agree on every row.)

Worked example: an “if not” rule

“If the patient is not fasting, then the glucose result is flagged.” Let f = “fasting” and r = “result flagged,” so the rule is $\neg f \rightarrow r$.

$$\begin{aligned} \neg f \rightarrow r &= \neg(\neg f) \vee r & (p \rightarrow q = \neg p \vee q, \text{ with } p = \neg f) \\ &= f \vee r & (\text{double negation}) \end{aligned}$$

The rule says: the patient was fasting, or the result is flagged. An “if not ..., then ...” rule is really an OR. The truth table agrees, with p in the role of f and q in the role of r :

p	q	$\neg p$	$\neg p \rightarrow q$	$p \vee q$
T	T	F	T	T
T	F	F	T	T
F	T	T	T	T
F	F	T	F	F

Worked example: three inputs, and the right way to nest implications

“If a patient is diabetic, then, if they are also hypertensive, they are on blood-pressure medication.” The natural symbols are $d \rightarrow (h \rightarrow m)$. A simpler way to say it seems to be “If a patient is diabetic and hypertensive, then they are on blood-pressure medication,” which is $(d \wedge h) \rightarrow m$. Are the two equivalent? Build both, eight rows.

d	h	m	$h \rightarrow m$	$d \rightarrow (h \rightarrow m)$	$d \wedge h$	$(d \wedge h) \rightarrow m$
T	T	T	T	T	T	T
T	T	F	F	F	T	F
T	F	T	T	T	F	T
T	F	F	T	T	F	T
F	T	T	T	T	F	T
F	T	F	F	T	F	T
F	F	T	T	T	F	T
F	F	F	T	T	F	T

The 5th and 7th columns agree on all eight rows, so the two statements are logically equivalent. Both are false only in row 2: a patient who is diabetic, hypertensive, and *not* on medication, the one patient who breaks the rule. This also settles the warning of Section 4.3. If you ever need to chain implications, the grouping $p \rightarrow (q \rightarrow r)$ is the one that means “if p and q , then r ”; the other grouping means something else entirely.

Exercise 4.4

Build the truth table for

$$(p \wedge q) \vee (\neg p \wedge \neg q).$$

Then answer: which connective from Section 3 is this equivalent to? Use these columns:

p	q	$p \wedge q$	$\neg p$	$\neg q$	$\neg p \wedge \neg q$	full expression
T	T					
T	F					
F	T					
F	F					

Solution to Exercise 4.4

Fill the columns from left to right. $p \wedge q$ is true only in the first row; $\neg p$ and $\neg q$ flip the input columns; $\neg p \wedge \neg q$ is true only where both of those are true, which is the last row. The full expression is the OR of the $p \wedge q$ and $\neg p \wedge \neg q$ columns, so a row is TRUE when either of those is true.

p	q	$p \wedge q$	$\neg p$	$\neg q$	$\neg p \wedge \neg q$	full expression
T	T	T	F	F	F	T
T	F	F	F	T	F	F
F	T	F	T	F	F	F
F	F	F	T	T	T	T

The final column is T, F, F, T, which is the table for **bi-implication**:

$$(p \wedge q) \vee (\neg p \wedge \neg q) = p \leftrightarrow q.$$

This makes sense: the expression says “both true, or both false,” which is exactly “ p and q match.”

Exercise 4.5

Exclusive OR is “ p or q , but not both.” Translate that phrase into symbols using only $\vee$, $\wedge$, and $\neg$, and prove with a truth table that it matches the XOR column. Then show that $\neg(p \leftrightarrow q)$ matches it too.

Solution to Exercise 4.5

“ p or q ” is $p \vee q$; “but” is AND; “not both” is $\neg(p \wedge q)$. So the translation is $(p \vee q) \wedge \neg(p \wedge q)$. Build it inside out and compare with XOR from the summary table:

p	q	$p \vee q$	$p \wedge q$	$\neg(p \wedge q)$	$(p \vee q) \wedge \neg(p \wedge q)$	$p \text{ XOR } q$
T	T	T	T	F	F	F
T	F	T	F	T	T	T
F	T	T	F	T	T	T
F	F	F	F	T	F	F

The last two columns are both F, T, T, F, so they are equivalent. The “not both” piece does exactly one job: it switches off the first row, the only row where inclusive and exclusive OR differ.

For the second claim, negate the IFF column:

p	q	$p \leftrightarrow q$	$\neg(p \leftrightarrow q)$	$p \text{ XOR } q$
T	T	T	F	F
T	F	F	T	T
F	T	F	T	T
F	F	T	F	F

Again the last two columns match. The one-liners predicted this: IFF is true when the values match, XOR is true when they differ, so each is the negation of the other.

Exercise 4.6

The study criterion contains $d \wedge (h \vee o)$. A colleague writes the query as $(d \wedge h) \vee (d \wedge o)$ instead: “diabetic and hypertensive, or diabetic and obese.” Is that the same cohort? Decide with a truth table.

Solution to Exercise 4.6

Build both sides on the 8 rows for d, h, o .

d	h	o	$h \vee o$	$d \wedge (h \vee o)$	$d \wedge h$	$d \wedge o$	$(d \wedge h) \vee (d \wedge o)$
T	T	T	T	T	T	T	T
T	T	F	T	T	T	F	T
T	F	T	T	T	F	T	T
T	F	F	F	F	F	F	F
F	T	T	T	F	F	F	F
F	T	F	T	F	F	F	F
F	F	T	T	F	F	F	F
F	F	F	F	F	F	F	F

The 5th and 8th columns agree on every row, so yes: the colleague’s query selects exactly the same patients. Both are true precisely in the first three rows, the diabetic patients with at least one of the other two conditions. Contrast this with $(d \wedge h) \vee o$ from Section 4.4, which is *not* the same cohort: there d was attached to h only, so obese patients without diabetes slipped in. The colleague’s version keeps d attached to *both* alternatives, which is why it works.

5 De Morgan's Laws

5.1 Negating AND and OR

How do you negate a compound condition? The naive guess, “flip everything,” is wrong in an interesting way: you do flip each piece, but you also have to flip the connective.

De Morgan's Laws

$$\neg(p \wedge q) = \neg p \vee \neg q$$

$$\neg(p \vee q) = \neg p \wedge \neg q$$

In words: push the negation inward, and **the connective flips**. AND becomes OR, and OR becomes AND.

For example, “It is not the case that I am both hungry and tired” means the same as “I am not hungry, or I am not tired.” To make “hungry and tired” false, it is enough for *either* one to fail.

The reason the connective flips is worth saying out loud, because it is the whole law:

- To deny an AND, it is enough to show that *one* part fails. So the denial is an **OR** of failures: $\neg p \vee \neg q$.
- To deny an OR, you must show that *every* part fails. So the denial is an **AND** of failures: $\neg p \wedge \neg q$.

Watch out: the naive flip

Flipping the pieces but keeping the connective gives a different statement. Compare $\neg(p \wedge q)$ with the naive $\neg p \wedge \neg q$:

p	q	$\neg(p \wedge q)$	$\neg p \wedge \neg q$
T	T	F	F
T	F	T	F
F	T	T	F
F	F	T	T

They disagree in rows 2 and 3. Row 2 is a diabetic patient without hypertension: they are certainly not “both diabetic and hypertensive,” yet “not diabetic and not hypertensive” is false for them. The naive version loses everyone who is missing just one of the two.

Worked example: an everyday OR

“It is not true that the patient has a fever or a cough.” With f = fever and c = cough, this is $\neg(f \vee c)$, and the second law gives

$$\neg(f \vee c) = \neg f \wedge \neg c.$$

The patient has no fever **and** no cough. Denying “fever or cough” rules out both symptoms, not just one.

Worked example: the outside of a range

In Section 3 the range “aged 18 to 65 inclusive” became $(\text{age} \geq 18) \wedge (\text{age} \leq 65)$. Who is *outside* the range? Apply the first law, then negate each comparison exactly, boundary included:

$$\begin{aligned}\neg((\text{age} \geq 18) \wedge (\text{age} \leq 65)) &= \neg(\text{age} \geq 18) \vee \neg(\text{age} \leq 65) && \text{(De Morgan)} \\ &= (\text{age} < 18) \vee (\text{age} > 65) && \text{(negating each comparison)}\end{aligned}$$

Check the four ages from before:

Age	in range	age < 18	age > 65	outside the range
17	F	T	F	T
40	T	F	F	F
65	T	F	F	F
70	F	F	T	T

Every age lands in exactly one of the two columns “in range” and “outside the range,” which is what a negation must do. The outside of a range is an OR, the same OR that appeared in the “under 18 and over 65” warning of Section 3.

Worked example: not meeting either threshold

In Section 3 a patient counted as diabetic if $f \vee a$: fasting glucose at least 126 mg/dL, or A1c at least 6.5%. Who does *not* count? By the second law,

$$\neg(f \vee a) = \neg f \wedge \neg a = (\text{FPG} < 126) \wedge (\text{A1c} < 6.5).$$

A patient escapes the definition only when **both** values are below their thresholds. Of the four patients in that example, only patient 4 (98 mg/dL and 5.4%) falls outside the definition, matching the one F in the $f \vee a$ column.

Worked example: negating a query with three conditions

A query keeps WHERE age ≥ 18 AND consent AND NOT dialysis. Which rows does it throw away? With a, c, k for the three basic statements, the query is $a \wedge c \wedge \neg k$. De Morgan’s law is stated for two pieces, but a chain is just two ANDs in a row (Section 4), so apply it twice:

$$\begin{aligned}\neg(a \wedge c \wedge \neg k) &= \neg((a \wedge c) \wedge \neg k) && \text{(the chain, grouped)} \\ &= \neg(a \wedge c) \vee \neg(\neg k) && \text{(De Morgan on the outer AND)} \\ &= (\neg a \vee \neg c) \vee k && \text{(De Morgan again; double negation)} \\ &= \neg a \vee \neg c \vee k && \text{(associative)}\end{aligned}$$

As a query:

WHERE age < 18 OR NOT consent OR dialysis

Three changes happened at once: every AND became OR, every piece was negated, and the double negative on dialysis collapsed. The same holds for a chain of any length: **the negation of “all of these” is “at least one of these fails.”**

5.2 Proving De Morgan's First Law

Claim: $\neg(p \wedge q) = \neg p \vee \neg q$. By the definition of logical equivalence, it is enough to build both truth tables and compare.

p	q	$p \wedge q$	$\neg(p \wedge q)$	$\neg p$	$\neg q$	$\neg p \vee \neg q$
T	T	T	F	F	F	F
T	F	F	T	F	T	T
F	T	F	T	T	F	T
F	F	F	T	T	T	T

The 4th and 7th columns match on every row, so the two statements are logically equivalent. ■

The second law is proved the same way.

Exercise 5.1

Prove De Morgan's second law, $\neg(p \vee q) = \neg p \wedge \neg q$, by building both truth tables.

Solution to Exercise 5.1

Compute $p \vee q$ and negate it for the left side; compute $\neg p$ and $\neg q$ and AND them for the right side.

p	q	$p \vee q$	$\neg(p \vee q)$	$\neg p$	$\neg q$	$\neg p \wedge \neg q$
T	T	T	F	F	F	F
T	F	T	F	F	T	F
F	T	T	F	T	F	F
F	F	F	T	T	T	T

The 4th and 7th columns are both F, F, F, T, so the two sides are logically equivalent. ■

In words: “ p or q ” is false only when both are false, and “not p and not q ” says precisely that both are false.

5.3 Why We Care

Go back to the study. Suppose the eligibility rule is *diabetic* $\wedge$ *hypertensive*. Who is **ineligible**? By De Morgan's first law,

$$\neg(\text{diabetic} \wedge \text{hypertensive}) = \neg \text{diabetic} \vee \neg \text{hypertensive}.$$

The ineligible patients are not “the patients who are neither”; they are the patients who are **missing either one**. A patient who is diabetic but not hypertensive is ineligible, and so is one who is hypertensive but not diabetic. Getting this wrong in a database query is a very easy mistake to make, and an expensive one.

Worked example: the right and wrong ineligible lists

Run the eligibility rule $d \wedge h$, the correct negation $\neg d \vee \neg h$, and the tempting wrong one $\neg d \wedge \neg h$ (“neither”) on the notebook's eight patients.

Patient	d	h	eligible: $d \wedge h$	correct: $\neg d \vee \neg h$	wrong: $\neg d \wedge \neg h$
P1	T	T	T	F	F
P2	T	F	F	T	F
P3	F	T	F	T	F
P4	T	T	T	F	F
P5	F	F	F	T	T
P6	T	F	F	T	F
P7	F	T	F	T	F
P8	F	F	F	T	T

Two patients (P1 and P4) are eligible. The correct negation lists six as ineligible; the wrong one lists only two (P5 and P8). There is a quick sanity check that catches the mistake without any theory: **every patient must be either eligible or ineligible, never both and never neither**, so the two lists must add up to the whole clinic. With the correct negation, $2 + 6 = 8$. With the wrong one, $2 + 2 = 4$, and the four patients who fell through the cracks (P2, P3, P6, P7) are exactly the ones with one condition but not the other.

Worked example: finding the patients who break a rule

In Section 3 we audited “if potassium is above 5.5, then the chart is flagged” by looking for rows where the antecedent is true and the consequent is false. De Morgan’s laws show that this recipe is not a trick but an equivalence:

$$\begin{aligned}
 \neg(p \rightarrow q) &= \neg(\neg p \vee q) & (p \rightarrow q &= \neg p \vee q) \\
 &= \neg(\neg p) \wedge \neg q & (\text{De Morgan's second law}) \\
 &= p \wedge \neg q & (\text{double negation})
 \end{aligned}$$

The truth table confirms it:

p	q	$p \rightarrow q$	$\neg(p \rightarrow q)$	$\neg q$	$p \wedge \neg q$
T	T	T	F	F	F
T	F	F	T	T	T
F	T	T	F	F	F
F	F	T	F	T	F

So the query for the violations of *any* if-then rule is: **antecedent AND NOT consequent**. For the potassium rule that is “potassium above 5.5 and chart not flagged,” which picks out record C and nothing else.

Worked example: a “never both” rule

“If a patient is pregnant, then the patient is not enrolled”: $g \rightarrow \neg e$. Rewrite it:

$$\begin{aligned}
 g \rightarrow \neg e &= \neg g \vee \neg e & (p \rightarrow q &= \neg p \vee q) \\
 &= \neg(g \wedge e) & (\text{De Morgan's first law, read right to left})
 \end{aligned}$$

The rule says no patient is **both** pregnant and enrolled. Again the table agrees, with p for g and q for e :

p	q	$\neg q$	$p \rightarrow \neg q$	$p \wedge q$	$\neg(p \wedge q)$
T	T	F	F	T	F
T	F	T	T	F	T
F	T	F	T	F	T
F	F	T	T	F	T

The two forms are the same rule. The second is often the easier one to check in data, because it names the forbidden combination directly: search for pregnant AND enrolled, and expect to find no one.

Exercise 5.2

Negate the full cohort criterion:

$$\text{diabetic} \wedge (\text{hypertensive} \vee \text{obese}) \wedge \neg \text{pregnant}.$$

Push the negation all the way in, so that no $\neg$ is left sitting outside a parenthesis.

Solution to Exercise 5.2

Write $d = \text{diabetic}$, $h = \text{hypertensive}$, $o = \text{obese}$, $g = \text{pregnant}$. Push the negation in one layer at a time. First, the outer chain of ANDs flips to a chain of ORs (De Morgan's first law, applied to all three pieces):

$$\neg(d \wedge (h \vee o) \wedge \neg g) = \neg d \vee \neg(h \vee o) \vee \neg(\neg g).$$

Two negations are still sitting outside something. Apply De Morgan's second law to $\neg(h \vee o)$, and use $\neg\neg g = g$:

$$\neg d \vee \neg(h \vee o) \vee \neg(\neg g) = \neg d \vee (\neg h \wedge \neg o) \vee g.$$

In words, a patient is **ineligible** if they are not diabetic, **or** they are neither hypertensive nor obese, **or** they are pregnant.

Notice that the inner $\vee$ flipped to an $\wedge$ while the outer $\wedge$ s flipped to $\vee$ s: every connective changes, at every level.

As a spot check, evaluate both the original criterion and the negated form on the notebook's eight patients:

Patient	d	h	o	g	$\neg d$	$\neg h \wedge \neg o$	g	ineligible	eligible
P1	T	T	F	F	F	F	F	F	T
P2	T	F	T	F	F	F	F	F	T
P3	F	T	T	T	T	F	T	T	F
P4	T	T	F	F	F	F	F	F	T
P5	F	F	F	T	T	T	T	T	F
P6	T	F	T	F	F	F	F	F	T
P7	F	T	T	F	T	F	F	T	F
P8	F	F	F	F	T	T	F	T	F

The “ineligible” column is the OR of the three middle columns, and on every row it is the exact opposite of “eligible”: four patients are in, four are out, and nobody is in both or neither. Eight patients are only a spot check; a full proof would compare the two sides on all $2 \times 2 \times 2 \times 2 = 16$ rows of the four-input truth table, which is exactly the kind of job to hand to a computer.

5.4 Extra Practice

Exercise 5.3

Negate “The patient is over 65 or has chronic kidney disease,” pushing the negation all the way in. Say the result in plain English.

Solution to Exercise 5.3

Let a = “over 65” and k = “has chronic kidney disease.” The statement is $a \vee k$, and by De Morgan’s second law

$$\neg(a \vee k) = \neg a \wedge \neg k.$$

Now negate each piece exactly: “not over 65” is “at most 65.” So the negation is: “The patient is **at most 65 and** does not have chronic kidney disease.” Denying an OR requires both parts to fail.

Exercise 5.4

A triage rule sends a patient to the isolation area if “they have a fever and a cough, or they have a positive flu test.” With f , c , t for the three statements, the rule is $(f \wedge c) \vee t$. Write the condition for a patient who is *not* sent to isolation, with no $\neg$ left outside a parenthesis, and say it in words.

Solution to Exercise 5.4

Work from the outside in.

$$\begin{aligned}\neg((f \wedge c) \vee t) &= \neg(f \wedge c) \wedge \neg t && \text{(De Morgan's second law on the outer OR)} \\ &= (\neg f \vee \neg c) \wedge \neg t && \text{(De Morgan's first law on the inner AND)}\end{aligned}$$

In words: the patient is missing the fever or the cough (at least one of them), **and** does not have a positive flu test. Both parts are needed: a patient with no cough but a positive test is still sent to isolation. As in the cohort exercise, each connective flipped: the outer OR became AND, and the inner AND became OR.

Exercise 5.5

A data filter keeps `age >= 40 AND (bmi >= 30 OR smoker)`. Write the filter that keeps exactly the rows this one discards.

Solution to Exercise 5.5

With a = “age ≥ 40 ,” b = “BMI ≥ 30 ,” and s = “smoker,” the filter is $a \wedge (b \vee s)$. Negate:

$$\begin{aligned}\neg(a \wedge (b \vee s)) &= \neg a \vee \neg(b \vee s) && \text{(De Morgan's first law)} \\ &= \neg a \vee (\neg b \wedge \neg s) && \text{(De Morgan's second law)}\end{aligned}$$

Negating each comparison exactly ($\geq$ becomes $<$), the new filter is

$$\text{age} < 40 \text{ OR } (\text{bmi} < 30 \text{ AND NOT smoker})$$

Here the AND-first default of SQL and Python would happen to produce the intended grouping even without the parentheses, but write them anyway: the next person to read the query should not have to know the default to know what it means.

Exercise 5.6

Using the policy from Section 4.6, write a query condition that finds the patients who violate each rule:

- (a) $w \rightarrow i$: every patient on warfarin has an INR recorded;
- (b) $b \rightarrow \neg w$: no patient with a prior major bleed is on warfarin.

Solution to Exercise 5.6

The violations of $p \rightarrow q$ are $\neg(p \rightarrow q) = p \wedge \neg q$: antecedent AND NOT consequent.

- (a) $\neg(w \rightarrow i) = w \wedge \neg i$: on warfarin **and** no INR recorded.
- (b) $\neg(b \rightarrow \neg w) = b \wedge \neg(\neg w) = b \wedge w$: a prior major bleed **and** on warfarin. The double negation collapses, and the query names the forbidden combination directly.

The patient checked in Section 4.6 ($w = \text{T}$, $i = \text{T}$, $b = \text{T}$) turns up in query (b) and not in query (a), matching what we found there.

6 Converse, Inverse, and Contrapositive

6.1 Three Relatives of an Implication

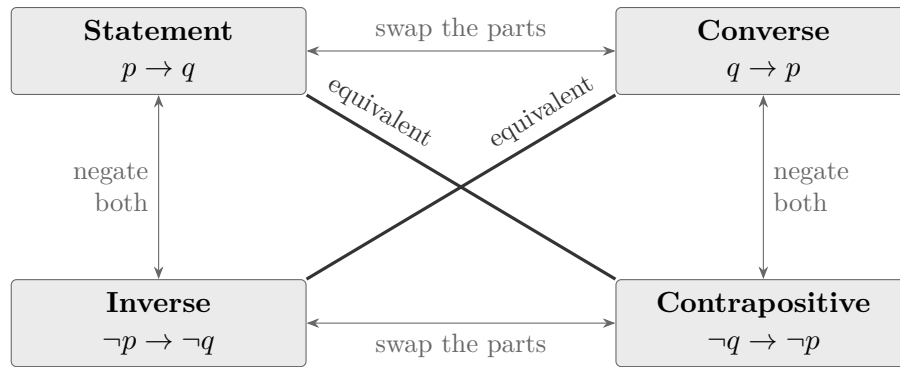
Start with an implication $p \rightarrow q$. There are three natural ways to rearrange it, by swapping the two sides, negating both, or doing both:

Statement	$p \rightarrow q$
Converse	$q \rightarrow p$
Inverse	$\neg p \rightarrow \neg q$
Contrapositive	$\neg q \rightarrow \neg p$

An example makes the difference clear.

- **Statement:** If it is raining, then the ground is wet.
- **Converse:** If the ground is wet, then it is raining. *Not the same:* a sprinkler also wets the ground.
- **Inverse:** If it is not raining, then the ground is not wet. *Not the same,* for the same reason.
- **Contrapositive:** If the ground is not wet, then it is not raining. *The same:* if rain always wets the ground, dry ground rules out rain.

The three rearrangements fit together as the corners of a square. Moving along an edge changes the meaning; the diagonals join the pairs that always agree, as the next subsection proves.



To form each one mechanically:

- **converse:** swap the two parts;
- **inverse:** negate both parts, keep the order;
- **contrapositive:** swap the parts *and* negate both.

Worked example: a statement about numbers

Start from “If $x > 5$, then $x > 3$,” which Section 3 showed is true for every x . The negation of $x > 5$ is $x \leq 5$, and of $x > 3$ is $x \leq 3$ (the boundaries switch sides).

Version	Sentence	True for every x ?
Statement	If $x > 5$, then $x > 3$.	Yes.
Converse	If $x > 3$, then $x > 5$.	No: $x = 4$.
Inverse	If $x \leq 5$, then $x \leq 3$.	No: $x = 4$.
Contrapositive	If $x \leq 3$, then $x \leq 5$.	Yes.

At $x = 4$ the converse has a true antecedent ($4 > 3$) and a false consequent ($4 > 5$ fails), and so does the inverse ($4 \leq 5$ holds, $4 \leq 3$ fails). The contrapositive is true for every x : any number at most 3 is certainly at most 5. It is no coincidence that the same $x = 4$ breaks both the converse and the inverse; the next subsection explains why.

Worked example: a clinical fact

“If a patient has type 1 diabetes, then the patient needs insulin.”

- **Converse:** If a patient needs insulin, then the patient has type 1 diabetes. *False:* many patients with type 2 diabetes also need insulin.
- **Inverse:** If a patient does not have type 1 diabetes, then the patient does not need insulin. *False,* with the same counterexample: a type 2 patient on insulin.
- **Contrapositive:** If a patient does not need insulin, then the patient does not have type 1 diabetes. *True,* and it says exactly what the original says.

Worked example: a protocol rule

“If the patient is pregnant, then the patient is excluded.”

- **Converse:** If the patient is excluded, then the patient is pregnant. *No:* patients are excluded for many reasons, such as dialysis or age.
- **Inverse:** If the patient is not pregnant, then the patient is not excluded. *No,* for the same reason.
- **Contrapositive:** If the patient is not excluded (that is, enrolled), then the patient is not pregnant. *Yes.*

The contrapositive is the form a data check naturally takes: go through the enrolled patients and confirm that none of them is pregnant.

Worked example: a rule with a compound antecedent

“If a patient is diabetic and hypertensive, then the patient is on blood-pressure medication”: $(d \wedge h) \rightarrow m$. Negating the antecedent $d \wedge h$ now needs De Morgan’s first law.

- **Converse:** $m \rightarrow (d \wedge h)$. “If a patient is on blood-pressure medication, then they are diabetic and hypertensive.” *Not equivalent:* plenty of treated patients are not diabetic.
- **Inverse:** $\neg(d \wedge h) \rightarrow \neg m$, which is $(\neg d \vee \neg h) \rightarrow \neg m$. “If a patient is not diabetic or not hypertensive, then they are not on blood-pressure medication.” *Not equivalent,* with the same counterexample.
- **Contrapositive:** $\neg m \rightarrow \neg(d \wedge h)$, which is $\neg m \rightarrow (\neg d \vee \neg h)$. “If a patient is not on blood-pressure medication, then they are not diabetic or not hypertensive.” *Equivalent.*

The tempting shortcut “if not on medication, then not diabetic **and** not hypertensive” is the naive flip from Section 5, and it is wrong. Test it on an untreated patient with diabetes only ($d = T$, $h = F$, $m = F$). The original rule is fine for them, $(T \wedge F) \rightarrow F = F \rightarrow F = T$, but the shortcut says $T \rightarrow (F \wedge T) = T \rightarrow F = F$, accusing them of breaking a rule they kept.

Watch out: two changes, each done correctly

The contrapositive needs *two* changes: swap *and* negate. Doing only the swap gives the converse; doing only the negation gives the inverse. And each negation must be an exact negation. The contrapositive of “if the temperature is above 38.0 °C, then a fever alert fires” is “if no fever alert fires, then the temperature is **at most** 38.0 °C,” not “below 38.0 °C.”

6.2 Only the Contrapositive Is Equivalent

A truth table settles it.

p	q	$p \rightarrow q$ <i>statement</i>	$q \rightarrow p$ <i>converse</i>	$\neg p \rightarrow \neg q$ <i>inverse</i>	$\neg q \rightarrow \neg p$ <i>contrapositive</i>
T	T	T	T	T	T
T	F	F	T	T	F
F	T	T	F	F	T
F	F	T	T	T	T

Read off the table

- The statement and its **contrapositive** match: they are always equivalent.
- The converse and the **inverse** match each other, but neither matches the original.

Confusing a statement with its converse is one of the most common reasoning errors there is. “If the patient has the disease, the test is positive” does not give you “the test was positive, so the patient has the disease”; that is exactly the converse, and it ignores false positives.

The contrapositive, on the other hand, can be trusted. If a test *always* comes back positive when the disease is present, then its contrapositive says: if the test is negative, the patient does not have the disease. So of the two tempting conclusions from such a test, the one drawn from a *negative* result follows from the rule, and the one drawn from a *positive* result does not.

A second proof, without a table

The equivalences from Section 4 give a proof by rewriting, one justified step at a time:

$$\begin{aligned}
 \neg q \rightarrow \neg p &= \neg(\neg q) \vee \neg p && (p \rightarrow q = \neg p \vee q) \\
 &= q \vee \neg p && (\text{double negation}) \\
 &= \neg p \vee q && (\text{commutative}) \\
 &= p \rightarrow q && (p \rightarrow q = \neg p \vee q, \text{ read right to left})
 \end{aligned}$$

This proof also explains *why*: the statement and its contrapositive both say “ p is false or q is true,” just in different words.

Why do the converse and the inverse match each other? Because the inverse is the contrapositive of the converse. Take the converse $q \rightarrow p$, swap and negate, and you get $\neg p \rightarrow \neg q$, the inverse. Every implication matches its own contrapositive, so the converse and inverse must match. That is why the single value $x = 4$ broke both of them in the number example.

When the converse is true too

Sometimes the converse of a true statement happens to be true as well. “If a whole number is even, then it is divisible by 2” is true, and so is its converse. That is not because converses follow in general; each direction had to be checked on its own. When both directions do hold, the pair is exactly a bi-implication: “even if and only if divisible by 2,” the form every definition takes (Section 3).

Worked example: what can you conclude from a rule?

Suppose the rule “If a patient is on dialysis, then their potassium is checked daily” is always followed. For four different patients, we know one fact each. What follows?

What we know	What follows	Why
The patient is on dialysis.	Potassium is checked daily.	This is the rule itself.
Potassium is <i>not</i> checked daily.	The patient is not on dialysis.	The contrapositive, which is equivalent to the rule.
Potassium is checked daily.	Nothing.	Concluding “on dialysis” would use the converse; other patients may be checked daily too.
The patient is not on dialysis.	Nothing.	Concluding “not checked daily” would use the inverse; the rule says nothing about these patients.

Only two of the four facts lead anywhere, and they are exactly the two that use the rule or its contrapositive. The last two rows are the converse and inverse mistakes, in the wild.

6.3 Extra Practice

Exercise 6.1

“If a patient’s systolic blood pressure is above 180 mmHg, then the patient is sent to the emergency department.” Write the converse, the inverse, and the contrapositive, and say which of them is equivalent to the original.

Solution to Exercise 6.1

The exact negation of “above 180” is “at most 180.”

- **Converse:** If a patient is sent to the emergency department, then their systolic pressure is above 180.
- **Inverse:** If a patient’s systolic pressure is at most 180, then the patient is not sent to the emergency department.
- **Contrapositive:** If a patient is not sent to the emergency department, then their systolic pressure is at most 180.

Only the contrapositive is equivalent to the original. The converse and the inverse both fail for the same kind of patient: one sent to the emergency department for chest pain with a systolic pressure of 130.

Exercise 6.2

For whole numbers n , consider “If n is divisible by 4, then n is even.” Decide whether the statement, its converse, its inverse, and its contrapositive are each true for every n , and give a counterexample for each one that is not.

Solution to Exercise 6.2

- **Statement:** true. A number divisible by 4 is $4k = 2(2k)$ for some whole number k , which is even.
- **Converse:** “If n is even, then n is divisible by 4.” False: $n = 6$ is even but $6 \div 4$ is not a whole number.
- **Inverse:** “If n is not divisible by 4, then n is not even.” False: $n = 6$ again, not divisible by 4 yet even.
- **Contrapositive:** “If n is not even, then n is not divisible by 4.” True, since it is equivalent to the statement: an odd number cannot be a multiple of 4.

As predicted, one counterexample ($n = 6$, or equally $n = 2$ or $n = 10$) breaks the converse and the inverse together.

Exercise 6.3

A hospital follows the rule “If a patient is allergic to penicillin, then the chart shows an allergy alert.” What, if anything, can you conclude in each case?

- (a) Patient X’s chart shows no allergy alert.
- (b) Patient Y’s chart shows an allergy alert.
- (c) Patient Z is not allergic to penicillin.

Solution to Exercise 6.3

- (a) Patient X is not allergic to penicillin. This is the contrapositive: no alert $\rightarrow$ not allergic.
- (b) Nothing about penicillin. Concluding “Y is allergic to penicillin” would use the converse; the alert could be for a different allergy.
- (c) Nothing about the chart. Concluding “Z’s chart shows no alert” would use the inverse; Z may have an alert for some other allergy.

Exercise 6.4

“If a patient is under 18 or pregnant, then the patient is not enrolled.” With u , g , e for under 18, pregnant, and enrolled, write the contrapositive with no $\neg$ left outside a parenthesis, and say it in words.

Solution to Exercise 6.4

The rule is $(u \vee g) \rightarrow \neg e$. Swap and negate both parts, then simplify:

$$\begin{aligned}\neg(\neg e) &\rightarrow \neg(u \vee g) = e \rightarrow \neg(u \vee g) && \text{(double negation)} \\ &= e \rightarrow (\neg u \wedge \neg g) && \text{(De Morgan's second law)}\end{aligned}$$

In words: “If a patient is enrolled, then the patient is at least 18 and not pregnant.” This is the same rule seen from the enrolled patients’ side, and it is the version you would check against the enrollment list. Note the AND: an enrolled patient must pass *both* checks.

7 Review

What we covered

1. A **statement** carries one of exactly two truth values.
2. **Connectives** $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$ combine statements, and a **truth table** defines each one completely.
3. $\neg$ is **unary** and the rest are **binary**; $\wedge$ and $\vee$ are commutative and associative, while $\rightarrow$ is neither.
4. Statements are **logically equivalent** when their truth tables agree, and comparing truth tables is also how we prove it.
5. **De Morgan's laws** negate AND and OR by flipping the connective.
6. Only the **contrapositive** is equivalent to the original implication.

You should now be able to

- decide whether a sentence is a statement, and name statements with letters;
- evaluate $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$, and XOR, including the vacuous rows of $\rightarrow$;
- negate a comparison exactly, putting the boundary value on the correct side;
- build the truth table of a compound statement with two or three inputs, working from the inside out;
- translate a rule between English and symbols, with parentheses wherever AND meets OR;
- prove an equivalence (check every row) or disprove one (find one row);
- negate a compound condition with De Morgan's laws, and find the violations of an if-then rule;
- write the converse, inverse, and contrapositive of a rule, and know which one can be trusted.

Notebook: U1-1_Logic-1_ConnectivesAndTruthTables.ipynb

Next: Unit 1-2, **Sets:** the same logic applied to collections, where our study cohort finally becomes an actual set.